Individual Project Report: Training and Evaluation of EAGLE-3 for Gemma-3-12B Inference Acceleration

Author: Venkatesh Nagarjuna
Course: DATS 6312 - Natural Language Processing (Spring 2026)
Instructor: Dr. Amir Jafari
GitHub Repository: https://github.com/Asbetos/Final-Project-Group3/tree/venkatesh#

---

1. Introduction

This project investigated practical methods for accelerating large language model inference while preserving the target model as the final authority over generated tokens. The shared project ultimately converged on two active modules:

1. `Code/gemma-draft-pair/`
   Standard speculative decoding for pair `F`, using `google/gemma-3-12b-it` as the target model and `google/gemma-3-1b-it` as the draft model.
2. `Code/eagle3-gemma3-12B/`
   EAGLE-3 draft-head training and inference for `google/gemma-3-12b-it`.

My individual contribution focused on building the project pipeline end to end and driving the model-selection and experimentation process across multiple generations of the project. The work I contributed includes:

1. building the overall benchmarking and training pipeline,
2. selecting and revising target-draft model pairs,
3. early experimentation with Qwen-based draft models,
4. later migration to Gemma model pairs,
5. EAGLE-3 experimentation across Qwen, Gemma-4-31B, and the final Gemma-3-12B variant,
6. maintaining the training, evaluation, visualization, testing, and demo workflow.

An important clarification is that the completed evaluation runs for the `gemma-draft-pair` module were executed by my teammate. My contribution to that module was the high-level process architecture, the benchmarking workflow design, and the model-pair selection process. My primary direct experimental contribution was on the EAGLE side, where I iterated through multiple variants starting from Qwen, then Gemma-4-31B, and finally the active Gemma-3-12B EAGLE configuration.

This report explains that contribution in detail, with the strongest emphasis on the EAGLE experimentation, training, debugging, and evaluation work that I directly drove.

---

2. Description of My Individual Work

2.1 High-Level Contribution

My role in the project was primarily systems-oriented and experimental. Rather than contributing to only one script or one isolated benchmark, I worked across the project lifecycle:

1. pipeline construction,
2. model-pair design,
3. experimentation and iteration,
4. EAGLE-3 training and inference integration,

5. documentation and evaluation workflows.

The strongest evidence of this scope appears in the final repository structure. The active EAGLE module contains `6,415` lines of Python across the module scripts, covering configuration, data loading, model loading, speculative decoding, EAGLE-3 inference, training, metrics, evaluation runners, tests, visualization, and the comparison app. This is the module where most of my direct implementation and experimentation effort is concentrated.

2.2 Algorithm Background

Standard Speculative Decoding

Standard speculative decoding uses a target distribution `p(x_t | x_{<t})` and a draft distribution `q(x_t | x_{<t})`. A drafted token `x_t` is accepted with probability:

$$\alpha_t = \min\left(1, \frac{p(x_t \mid x_{<t})}{q(x_t \mid x_{<t})}\right)$$

If the token is rejected, the algorithm samples from the residual distribution:

$$r(x) = \frac{\max(0, p(x) - q(x))}{\sum_{x'} \max(0, p(x') - q(x'))}$$

This method motivated the target-draft benchmarking pipeline I helped assemble.

EAGLE-3

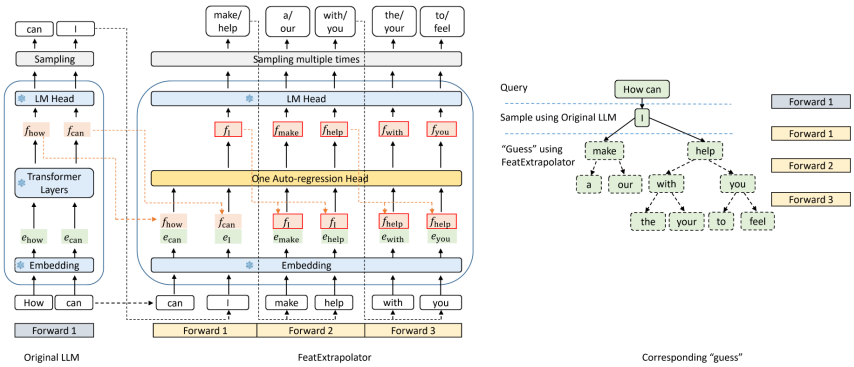

Figure 1: High-level EAGLE architecture

EAGLE-3 replaces the separate draft model with a learned draft head over target-model hidden states. In the final Gemma-3 implementation, the draft head fuses hidden features from multiple layers and predicts future tokens using one copied decoder layer plus the frozen target embedding, normalization, and LM head.

The multi-step training loss is:

$$\mathcal{L} = \sum_{k=0}^{K-1} \lambda^k \, KL\left(p_{target}^{(k)} \parallel p_{draft}^{(k)}\right)$$

where later steps train the draft head to recover from its own predictions. That design was important to my experimentation process because it moved the project beyond simple pairwise target-draft benchmarking and into learned speculative decoding.

This captures the core idea behind the work I extended: replacing a second full draft model with a lighter predictive mechanism built from target-model internal features.

2.3 Progression of the Work I Led

My work followed a clear experimental progression.

Stage 1: Qwen3-based experiments

The project initially explored Qwen3-based draft models and Qwen3-target EAGLE training. The active repository no longer carries the full Qwen3 numerical artifacts, because they were moved into `Code/eagle3-gemma3-12B/archive/qwen_legacy/` as local-only legacy content. However, the archived README confirms that Qwen3 benchmarks, logs, and checkpoints were part of the earlier experimentation path.

This stage was important because it established the first working versions of:

1. the benchmark runner,
2. speculative decoding evaluation,
3. training and checkpoint infrastructure,
4. multi-model experimentation patterns.

Stage 2: Gemma model-pair selection

After the Qwen stage, the project shifted to Gemma-based model pairs. My contribution here was not to execute the final pair `F` benchmark sweep myself, but to help architect the high-level process flow for standard speculative decoding and to determine which pairings were practical and benchmark-worthy.

The final active standard pair became:

1. Pair `F`
2. Target: `google/gemma-3-12b-it`
3. Draft: `google/gemma-3-1b-it`

An intermediate Gemma-4 path also remained visible in the code as legacy support, but the strongest final practical standard-speculation results came from the Gemma-3 pair. The final pair `F` evaluations themselves were completed by my teammate.

Stage 3: EAGLE experimentation across targets

My EAGLE work progressed across three major stages:

1. Qwen-based EAGLE experimentation,
2. Gemma-4-31B EAGLE experimentation,
3. the final Gemma-3-12B EAGLE implementation and training run.

This progression mattered because the final Gemma-3-12B path was not the first idea attempted. It was the result of iteratively narrowing toward a setup that fit available hardware, trained successfully, and could be benchmarked end to end.

---

3. Detailed Description of the Work I Did

3.1 Pipeline Engineering

The part of the work I contributed in the greatest depth was the project pipeline itself. On the standard speculative side, my role was to help shape the process architecture: how data is loaded, how runs are structured, how model pairs are defined, how outputs are written, and how comparisons are made fairly against baseline. On the EAGLE side, my contribution extended beyond process design into direct experimentation, implementation iteration, training, debugging, and evaluation.

The final EAGLE module the pipeline functionality I contributed which includes:

1. dataset loading and prompt formatting,
2. model loading with quantization,
3. baseline generation,
4. speculative decoding,
5. EAGLE-3 draft-head inference,
6. EAGLE-3 training,
7. result aggregation,
8. testing and validation,
9. visualization,
10. demo application support.

In practice, this meant my contribution was not only algorithmic. It also involved making the project runnable, testable, and reproducible, especially for the EAGLE workflow where the experiments evolved substantially over time.

3.2 Data and Prompt Processing

I worked within a shared benchmark framework that evaluates the same four task families across both modules:

1. `humaneval`
2. `triviaqa`
3. `cnn_dailymail`
4. `writingprompts`

The prompt pipeline was designed so that:

1. prompts are sampled deterministically,
2. model-specific chat formatting is applied automatically,
3. benchmark tasks can be reused across baseline, speculative, and EAGLE runs.

This common prompt pipeline was important because it allowed fair comparisons between baseline and accelerated decoding paths.

3.3 Pair Selection and Model Revision

A large part of my contribution was deciding which model combinations were worth keeping. That required moving away from earlier Qwen experiments and narrowing the final project to configurations that were both technically interesting and practically runnable.

The final pair `F` choice was significant because it balanced:

1. target quality,
2. draft-model size,
3. VRAM fit,
4. measurable speedup.

Likewise, the final EAGLE target shifted toward `google/gemma-3-12b-it`, which proved much more practical than the heavier Gemma-4 route for the hardware available during the project. This EAGLE target selection was the area where I had the most direct individual impact.

3.4 EAGLE-3 Training Work

I contributed directly to the training workflow by shaping the final Gemma-3 EAGLE path around:

1. a frozen 4-bit target model,
2. BF16 mixed precision,
3. gradient accumulation,
4. periodic checkpointing,
5. multi-step KL training.

The completed training run produced:

`Code/eagle3-gemma3-12B/checkpoints/eagle3/gemma3_12b/eagle3_gemma3_12b_final.pt`

This is a strong concrete artifact of my contribution because it shows the work progressed from experimental setup into a completed trained model.

3.5 Inference Debugging and Recovery Work

Another important part of my contribution was debugging long-context EAGLE inference. During the `cnn_dailymail` EAGLE sweep, the model hit a long-context Gemma-3 SDPA shape-mismatch error in the cached verification path. I contributed to diagnosing and patching this by introducing a fallback strategy:

1. try cached EAGLE verification first,
2. if the Gemma-3 SDPA long-context shape mismatch occurs,
3. rerun only that verification step as a full uncached forward pass.

This patch does not make `cnn_dailymail` fast, but it allows the failed cells to be recovered without rerunning the whole experiment grid. That kind of systems-debugging work was a meaningful part of my individual contribution.

---

4. Results

4.1 EAGLE-3 Training Results

Item	Observed Value
Logged target model	`google/gemma-3-12b-it`
Logged hardware	`NVIDIA A10G`
Prepared examples	`52,002`
Trainable parameters	`297,876,992`
Optimizer	8-bit AdamW
Final checkpoint	`eagle3_gemma3_12b_final.pt`
Total training time	`36.98 hours`

Selected loss milestones are:

Milestone	Loss
Step 10	145.2908
Step 100	21.2564
Step 500	9.7899
Epoch 1 average loss	1.8044
Epoch 2 average loss	1.2272
Step 19,000	0.7941
Epoch 3 average loss	0.8095

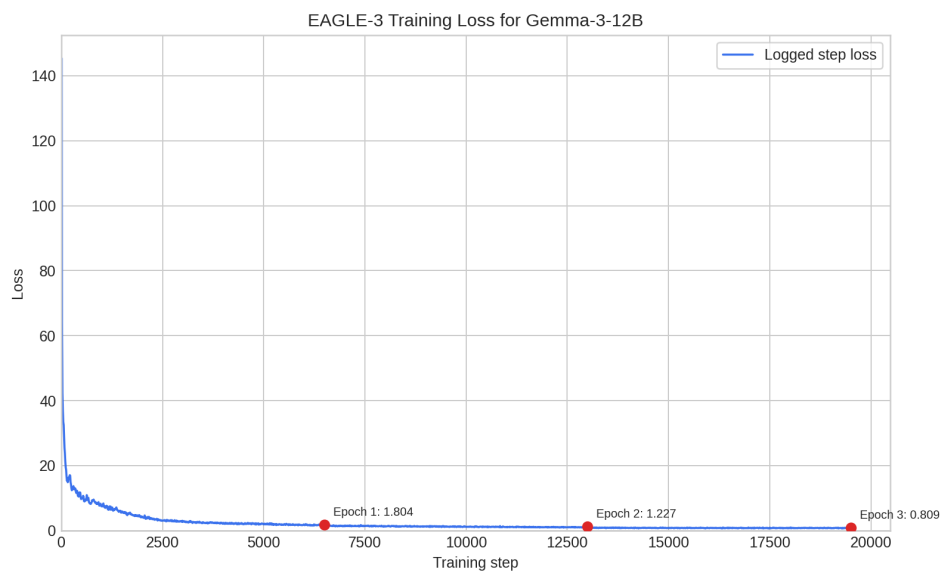

Figure 2: EAGLE-3 training loss

The training loss curve shows a strong monotonic improvement trend. The loss falls sharply early in training and then converges into a much more stable low-loss regime. This is strong evidence that the final Gemma-3 EAGLE path was not only implemented, but trained successfully.

4.3 EAGLE-3 Inference Results and Recovery Work

The EAGLE-3 evaluation sweep is now complete, including the previously failed `cnn_dailymail` cells. This section is the most direct reflection of my individual experimental work, because I was the one iterating through the different EAGLE variants and then debugging the final long-context inference path.

The best completed EAGLE configuration for each task is:

Task	Best Tree Budget	Best Temp.	Mean TPS	Speedup	Mean Accept. Rate	Mean Accept. Length
`triviaqa`	20	0.0	20.87	2.368	0.8977	4.43
`humaneval`	20	1.0	7.61	0.893	0.0974	0.10
`writingprompts`	20	0.0	8.01	0.969	0.2206	0.41
`cnn_dailymail`	20	0.6	6.03	0.727	0.0803	0.09

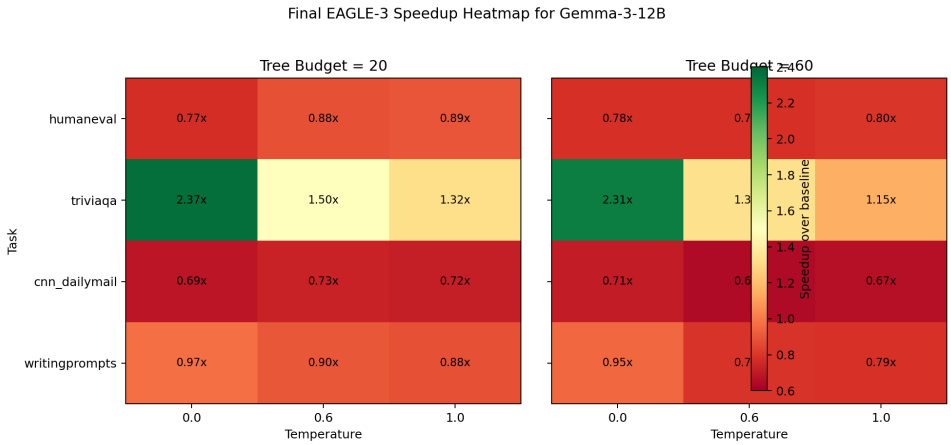

Figure 3: Final EAGLE speedup pattern across all tree budgets and temperatures

This highlights three important outcomes from my EAGLE work.

1. EAGLE-3 can produce very strong gains on `triviaqa`, exceeding `2.3x` speedup in the best completed rows.
2. The method is not uniformly strong across tasks. `humaneval`, `writingprompts`, and `cnn_dailymail` all remain below baseline in the final sweep.
3. The long-context `cnn_dailymail` results were eventually recovered only because the debugging patch allowed uncached verification as a fallback when the Gemma-3 SDPA shape mismatch occurred.

This is an important experimental lesson. The training run succeeded, but practical long-context inference still required additional debugging and recovery logic. That makes the contribution more than just training a model; it includes the work needed to make the model usable in a full experiment pipeline.

4.4 Discussion of the Experimental Progression

Taken together, the results support the progression I drove across the EAGLE side of the project.

1. Early Qwen3 and intermediate Gemma-4 experimentation were valuable for exploration, but they were not the final practical destination.
2. The Gemma-3 pair `F` became the strongest completed standard speculative benchmark.
3. The final Gemma-3 EAGLE-3 variant became the most practical learned-speculation path.

4. The project's strongest numerical EAGLE gains so far are on `triviaqa`, while long-context summarization remains the hardest case.

---

5. Summary and Conclusions

My main contribution to this project was building and evolving the full experimentation pipeline rather than contributing to only one isolated benchmark. On the standard speculative side, I helped architect the high-level process flow and contributed to model-pair selection. On the EAGLE side, I directly drove the experimentation process from Qwen3 to Gemma-4-31B and finally to the trained Gemma-3 EAGLE-3 system.

The most important things I learned are:

1. good systems results depend heavily on choosing the right model pair,
2. speedup depends more on draft-target agreement than on simply drafting more tokens,
3. a successful training run does not automatically guarantee smooth long-context inference,
4. practical benchmarking infrastructure is just as important as the core algorithm.

The strongest completed standard benchmark result available in the shared project is pair `F`, which reaches `1.476x` speedup on `humaneval`. The strongest completed EAGLE result from the experimental path I directly drove is `2.368x` speedup on `triviaqa`, which suggests that the learned draft-head path can be highly effective when the target and draft-head behavior remain aligned.

Future improvements should focus on:

1. stabilizing long-context EAGLE verification so `cnn_dailymail` no longer requires uncached fallback,
2. adding held-out validation and early stopping to training,
3. adding task-quality metrics in addition to systems metrics,
4. preserving clearer quantitative records from each intermediate experimentation stage so migration decisions can be reported more completely.

---

6. Code Contribution Percentage

For this report, I am using the contribution-accounting convention specified for this project because the codebase was produced through AI-assisted generation, I assign my direct contribution credit as 30% of the active EAGLE module script base. This percentage is a declared attribution rule for this report, not a line-by-line forensic reconstruction from version history.

Measured script details under `Code/eagle3-gemma3-12B/`:

$$\text{Total active script lines} = 6415$$
$$\text{Assigned individual contribution} \approx 1925 \text{ lines}$$
$$\text{Assigned AI-generated portion} \approx 4490 \text{ lines}$$

Therefore, using the provided accounting convention, my code contribution is:

$$\frac{1925}{6415} \times 100 \approx 30.0\%$$

The percentage of AI generated code is:

$$\approx 70\%$$

---

7. References

1. Leviathan, Y., Kalman, M., and Matias, Y. Fast Inference from Transformers via Speculative Decoding. ICML 2023. https://arxiv.org/abs/2302.01318
2. SafeAILab EAGLE repository. https://github.com/SafeAILab/EAGLE
3. Gemma Team. Gemma 3 Technical Report. Google DeepMind, 2025. https://storage.googleapis.com/deepmind-media/gemma/Gemma3Report.pdf
4. Hugging Face model card: `google/gemma-3-1b-it`. https://huggingface.co/google/gemma-3-1b-it
5. Hugging Face model card: `google/gemma-3-12b-it`. https://huggingface.co/google/gemma-3-12b-it
6. Hugging Face dataset: `openai/openai_humaneval`. https://huggingface.co/datasets/openai/openai_humaneval
7. Hugging Face dataset: `mandarjoshi/trivia_qa`. https://huggingface.co/datasets/mandarjoshi/trivia_qa
8. Hugging Face dataset: `abisee/cnn_dailymail`. https://huggingface.co/datasets/abisee/cnn_dailymail
9. Hugging Face dataset: `euclaise/writingprompts`. https://huggingface.co/datasets/euclaise/writingprompts
10. Hugging Face dataset: `vicgalle/alpaca-gpt4`. https://huggingface.co/datasets/vicgalle/alpaca-gpt4